(a) $D = \{1, 3, 4, 6\}$ is a dominating set, because 2 has an edge to 3, and 5 has an edge to 6.

$D' = \{1, 3, 5\}$ is a dominating set of size 3: 4 and 6 have edges to 5, and 2 has an edge to 3.

(b) The only stranger is 1. The only other root is 3, but it has children (2 and 5).

We will say that a vertex v is *dominated* by a set D if $v \in D$ or v has an edge to a vertex in D. A set is a dominating set if and only if every vertex is dominated by it.

Every stranger has degree 0, and hence to be dominated by a set, it must be in the set itself.

(c) 1, 2, 4, and 6 are leaves; both 3 and 5 have children and hence are not leaves. The only young parent is 5; the only other parent is 3 but it has a child (5) that is not a leaf.

Let y be a young parent and let D be a minimum-size dominating set. If $y \in D$ then we are done.

Suppose then that $y \notin D$ and let C be the set of children of y. We argue that $(D \setminus C) \cup \{ y \}$ is a dominating set: indeed, y and its children are dominated by $\{ y \}$, and all the other vertices are dominated by $D \setminus C$. Moreover, $|C| \geq 1$ (because y is a young parent) and $C \subseteq D$ (because all children of y have an edge only to $y \notin D$). It follows that $|(D \setminus C) \cup \{ y \}| = |D \setminus C| + 1 \leq (|D| - 1) + 1 = |D|$ and hence $(D \setminus C) \cup \{ y \}$ is also a minimum-size dominating set.

(d) We prove it by induction on the number of vertices in the tree.

In a tree with $n = 2$ vertices, one of the vertices is a root, and the other, its child, is a leaf. Therefore, the root is a young parent.

Suppose (the induction hypothesis) that every tree with at most n vertices contains a young parent. Consider a tree with $n + 1$ vertices; we now prove that it contains a young parent. If its root is a young parent then we are done. If, however, the root is not a young parent, then one of its children is the root of a tree with at most $n \geq 2$ vertices. By the induction hypothesis, this tree has a young parent, and we are done.

(e) **Algorithm description.** In order to design a correct algorithm for the problem, we consider a generalized concept of a marked forest and the suitably generalized concept of a dominating set. A *marked forest* is a forest in which some (possibly none) of the vertices are *marked*. We say that a set D is a *dominating set* in a marked forest if every vertex is either dominated by D or it is marked.

Consider the following (greedy) algorithm. Note that initially no vertex is marked.

1. First, set D to be the empty set.
2. While the forest is not empty, keep considering the following cases:
   i. If there is a marked leaf in the forest, then remove it from the forest.
   ii. If there are no marked leaves and there is a stranger in the forest, then add the stranger to D and remove it from the forest.
   iii. If there are no marked leaves and no strangers in the forest, then pick a young parent y, add y to D, mark the parent of y (if y has a parent), and remove the children of y and y itself from the forest.
3. Output set D.

Correctness. We argue that the algorithm returns a min-size dominating set. Note that in every execution of step 2., exactly one of the cases 2.i., 2.ii., or 2.iii. applies. If cases 2.i. or 2.ii. do not apply then case 2.iii. applies by part (d), and hence in every iteration of loop 2., the algorithm will make progress by reducing the size of the remaining marked forest, eventually producing an output.

The correctness of the algorithm if it performs step 2.iii. in its first iteration of loop 2. follows from the following fact. (The analogous facts for steps 2.i. and 2.ii. are easier to state and significantly easier to justify and hence are omitted.)

Fact 1. *Let F be a forest with no marked leaves and no strangers, let y be a young parent, and let F' be the marked forest obtained from F by removing y and its children, and marking the parent of y. If D' is a minimum-size dominating set in F' then $D' \cup \{ y \}$ is a minimum-size dominating set in F.*

Proof. Firstly, we argue that $D' \cup \{ y \}$ is a dominating set in F. Note that y, its children, and its parent are all dominated by $\{ y \}$. All the other vertices in F are also in F' and hence they are either marked or dominated by D' in F.

Secondly, we argue that if D is a dominating set in F then $|D' \cup \{ y \}| \leq |D|$, implying that $D' \cup \{ y \}$ is a minimum-size dominating set in F.

- We show that $D \cap F'$ is a dominating set in F': indeed, the parent of y is marked in F' and all the other vertices in F' are either marked or dominated by D in F'.
- Then $|D'| \leq |D \cap F'|$ because D' is a minimum-size dominating set in F'.
- Moreover, we have $|D \cap F'| < |D|$, because no leaf in F is marked, and hence at least one vertex among y and its children must be in D.
- It follows that $|D'| < |D|$, and hence $|D' \cup \{ y \}| = |D'| + 1 \leq |D|$. □

An efficient implementation. The algorithm can be implemented to run in time $O(n)$, where n is the number of vertices in the original forest. This will follow from $O(1)$-time procedures for selecting and removing leaves, strangers, and young parents. To facilitate this, consider the data structure that maintains:

- two counters for every vertex: number of children and number of children that are leaves;
- three lists: marked leaves, strangers, and young parents.

Whenever the values of the two counters become equal for a vertex, it is added to the list of young parents. A single post-order traversal of all trees allows to initialize the values of the counters and to construct the three lists in $O(n)$ time.

Note that we only need to clarify how to maintain the data structure efficiently when leaves are removed, because in step 2.iii., after removing all children of y, which are leaves, vertex y itself becomes a leaf.

Removing a leaf can be done in $O(1)$ time because it only may require:

- removing it from the list(s) of marked leaves and/or strangers;
- changing the counters for its parent and grandparent;
- if the leaf being removed is the only child, removing its parent from the list of young parents and adding it to the list(s) of marked leaves and/or strangers.